data management patterns

definiscono come i dati vengono roganizzati, letti e scritti in memoria

serial data management patterns

introduction

while control patterns dictate *when* things happen, data management patterns dictate *where* and how data is stored and accessed

- **definition**
  - data management deals with how data is allocated, shared, read, written and copied within a program
- **the serial foundation**
  - understanding how serial programs manage memory is a prerequisite for parallel programming
  - the mechanism used in serial code (pointers, stacks, heaps) have direct, often problematic, implications when ported to a parallel environment
- the four primary serial strategies are
  - random read and write
  - stack allocation
  - heap allocation
  - objects

random read and write pattern

this pattern represents the most flexible, yet potentially dangerous, way to access memory

- **mechanism**
  - memory locations are indexed using addresses, typically via **pointers**
    - this allows the program to jump to any location in memory to read or write data

capacità di accedere a qualsiasi locazione di memoria tramite un indirizzo (puntatore)

- **aliasing problem**
  - the most significant challenge with this pattern when moving to parallelism is **aliasing**
    - aliasing occurs when two different pointers refer to the same memory object
    - if a compiler or runtime cannot determine if pointer A and pointer B overlap, it can not safely execute operations on them in parallel
      - potential Read-After-Write or Write-After-Write conflicts
  - controlling this random access is key
    - techniques like **tiling** are essentially ways to impose structure on random reads/writes to ensure memory coalescing and cache efficiency

se due puntatori diversi puntano alla stessa locazione di memoria e due thread scrivono contemporaneamente usando questi puntatori, si verifica una race condition

Random Read and Write (Lettura/Scrittura Casuale):

- *In Seriale:* Usi puntatori per accedere a qualsiasi indirizzo di memoria. È la base della flessibilità.
- *Il problema nel Parallelo:* Quando porti questo pattern in parallelo, nasce il problema dell'**Aliasing** (due puntatori diversi che puntano allo stesso dato). Se non gestito, porta a Race Conditions . Quindi, studiamo il pattern "seriale" per capire che in parallelo dobbiamo coordinare questi accessi.

stack allocation pattern

stack allocation is a disciplined form of memory management

l'allocazione avviene in modalità LIFO (Last-In, First-Out) → è estremamente veloce e preserva la località dei dati

- **structure**
  - data is allocated and deallocated in a **Last-In, Fist-Out (LIFO)** manner
- **efficiency**
  - it is extremely efficient because allocation simply involves moving the stack pointer
    - it takes constant time regardless of the allocation size
- **locality**
  - stack allocation inherently preserves **spatial locality**
    - data declared together in a function tends to be used together and sits close in memory
- **parallel implication**
  - this is a "friendly" pattern for parallelism

il fatto che ogni thread riceve il proprio stack privato garantisce che le variabili locali siano isolate e thread-safe senza bisogno di sincronizzazione

  - when a program is parallelized, each thread usually gets its **own private stack**
    - this naturally provides **Thread Locality** (data isolation), preventing race conditions for local variables without requiring explicit locks

Stack Allocation:

- *In Seriale:* Allocazione LIFO (Last-In, First-Out) veloce e con ottima località (i dati usati di recente sono vicini).
- *L'adattamento Parallelo:* Per mantenere questa efficienza in parallelo, non possiamo condividere un unico stack. Ogni thread deve ricevere il **proprio stack privato**. Questo preserva la "thread locality" ed evita la necessità di lock costosi per ogni variabile locale.

heap allocation pattern

heap allocation handles dynamic data that does not fit the strict LIFO structure of the stack

- **structure**
  - data can be allocated or freed at any time, in any order
- **drawbacks**
  - it is slower and more complex than stack allocation due to the need to search for free memory blocks and manage fragmentation
- **parallel implication**
  - a standard global heap allocator becomes a massive bottleneck in parallel programs because multiple threads competing for malloc/new must lock the heap to prevent corruption
  - **solution**
    - to avoid this serialization, parallel programs use **Parallelized Heap Allocators**
    - these allocators maintain **separate memory pools** for each parallel worker (thread), allowing them to allocate memory concurrently without locking the global heap

Heap Allocation:

- *In Seriale:* Memoria dinamica globale (es. `malloc` / `new`) per dati che sopravvivono alle funzioni. È più lenta dello stack.
- *Il problema nel Parallelo:* Se più thread provano ad allocare memoria dallo stesso Heap contemporaneamente, devono competere per il lock dell'allocatore, creando un collo di bottiglia. La soluzione parallela è usare allocatori che mantengono **pool di memoria separati** per ogni worker.

objects pattern

the objects pattern refers to the standard **Object-Oriented Programming (OOP)** construct

- **definition**
  - language constructs that associate data (member variables) with the code (member functions) used to manipulate that data
- **parallel generalization**
  - in parallel programming models, objects are often generalized
    - the "object" becomes the unit of distribution or coherence
    - we must consider if an object is "thread-safe" (can its member functions be called by multiple threads simultaneously without corrupting its internal data?)

Objects:

- *In Seriale:* Associano dati e codice per manipolarli.
- *Il problema nel Parallelo:* Quando si parallelizza, bisogna decidere se un oggetto è condiviso (pericolo di race condition) o copiato/privato per ogni thread. I modelli di programmazione parallela devono generalizzare questo concetto per gestire la coerenza dei dati interni all'oggetto.

parallel data management patterns

introduction

data management patterns focus on the efficient organization, access and movement of data in parallel memory hierarchies

- while control patterns manage the flow of tasks
- **the critical challenge**
  - in parallel programming, *knowing when* data is shared and when it is private is critical to avoiding **race conditions**
- **locality**
  - these patterns are designed to assist with **data locality**
    - keeping data close to the processor that needs it to minimize latency

pack pattern

the **pack pattern** is used to compact data, typically after a filtering operations

- **definition**
  - it eliminates unused space in a collection
    - elements marked as "false" (or invalid) are discarded
    - the remaining elements are placed in a contiguous sequence, maintaing their original relative order

evita di sprecare banda di memoria e cicli di calcoli inutili

- **use case**
  - it is extremely useful when combined with the **Map** pattern
    - **ex.** after mapping a function that produces many "null" or invalid results, Pack consolidates the valid results
- **inverse operation**
  - the inverse pattern is **unpack**, which places elements back into their original locations
    - often inserting default values or gaps into the sequence

pipeline pattern

the **pipeline pattern** organizes data flow between tasks

- **definition**
  - it connects tasks in a **Producer-Consumer** relationship: data flows from one stage to the next
    - **ex.** Stage A produces data consumed by Stage B

connette i task in una catena di montaggio per cui i dati fluiscono attraverso vari stadi

- **structure**
  - while a linear pipeline is the basic form, pipelines can also be structured as a Directed Acyclic Graph (DAG)

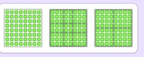
- **parallelism multiplier**
  - pipelines are most effective when combined with other patterns

mentre lo stadio B elabora il dato i, lo stadio A può elaborare contemporaneamente il dato i+1

  - they multiply the available parallelism by allowing different stages to process different data items simultaneously
    - **ex.** while Stage B processes Item 1, Stage A can start processing Item 2

geometric decomposition pattern

this pattern is fundamental to **domain decomposition** in scientific computing



- **definition**
  - it arranges data into sub-collections based on the geometry of the problem domain
- **types**
  - **non-overlapping**
    - the domain is sliced into distinct, separate chunks
  - **overlapping**
    - sub-domains share boundary regions which is crucial for **stencil** computations where a task need neighbors' data
- **key characteristic**
  - this pattern doesn't necessarily move data physically
    - it often provides a specific "view" or indexing logic over the existing data structure

gather pattern

the **Gather** pattern handles random read access patterns

legge dati da posizioni casuali di un array sorgente basandosi su un array di indici e li raggruppa in un array di destinazione contiguo `output[i] = input[indices[i]]`

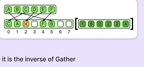
- **definition**
  - it **creates** a new collection of data by reading from an input collection based on a specific set of **indices**

è un'operazione sicura essendo di sola lettura

- **analogy**
  - think of it as a combination of a **Map** pattern and **Random Serial Reads**
- **structure**
  - **input**
    - a source data array and an array of indices
  - **output**
    - an array where `Output[i] = Input[Indices[i]]`
    - the output shares the shape of the *indices* collection, not necessarily the input data
- **optimization**
  - this is a key pattern for vectorization (*Intel AVX*) and for handling sparse data structures where dense packing is required for efficiency

scatter pattern

the **Scatter** pattern handles random write access patterns



scrive dati da un array contiguo verso posizioni sparse (casuali) in memoria, basandosi su indici `output[indices[i]] = input[i]`

- **definition**
  - it is the inverse of Gather
  - it writes elements to the output collection at positions specified by a set of indices
    - instead of reading from random locations
- **mechanism**
  - each element of the input is written to the output at the given index `Output[Indices[i]] = Input[i]`
- **the race condition danger**
  - if the index collection contains duplicates (i.e., two inputs map to the same output index), a **Race Condition** occurs
  - **conflict resolution**
    - to handle this safely, the pattern often requires specific conflict resolution rules
    - **ex.** Atomic Scatter, Permutation Scatter or Merge Scatter where values are summed/ combined rather then overwritten